

Programming Methodology - SE1012

Lab 4 - Top-Down Design with Functions



Using Linux machines

Access Lab 4 via GitHub:

<https://classroom.github.com/a/VkxX8oZD>

Outline of Today's Lab



- Top-Down Design
- Functions without Arguments and with Input Arguments
- Using Library Functions
- Function Prototypes
- Activity 1: Binary to Decimal Converter
- Activity 2: Quadratic Equation Solver

Top-Down Design

Use top-down design to break the problem into subproblems. Example: Design a program to calculate the area and circumference of a circle.

1. Get the radius from the user.
2. Calculate the area using a function.
3. Calculate the circumference using a function.
4. Display the results.

Implement these subproblems as separate functions and integrate them into your main program.

Functions without and with Input Arguments

Functions without Arguments:

Simple functions that do not require any input parameters. Example: Write a program that includes a function `printWelcomeMessage` that prints `Welcome to C Programming!` when called. Implement and call this function in your main program.

```
void printWelcomeMessage() {  
    printf("Welcome to C Programming!\n");  
}
```



Functions with Input Arguments:

Enable the passing of information into functions, making them adaptable to various tasks. Example: Write a program that includes a function `calculateCircleArea` that takes the radius `r` as an input argument and returns the area of the circle. Use the formula: $area = \pi * r^2$.

```
double calculateCircleArea(double radius) {  
    return 3.14159 * radius * radius;  
}
```



Using Library Functions

Using Library Functions:

Predefined functions provided by C for common tasks are called as library functions. Example: Write a program that uses the `math` library functions `sqrt` and `pow` to calculate the square root and power of a given number. For a given number, calculate its square root, and the number raised to the power of 3.

```
#include <math.h>

double calculateSquareRoot(double number) {
    return sqrt(number);
}

double calculatePowerOfThree(double number) {
    return pow(number, 3);
}
```



Function Prototypes

Function Prototypes

Declare the functions before they are used in the main function.

Example:

```
#include <stdio.h>
#include <math.h>

// Function prototypes
double calculateSquareRoot(double number);
double calculatePowerOfThree(double number);

int main() {
    double number;

    // Get user input
    printf("Enter a number: ");
    scanf("%lf", &number);

    // Calculate the square root and power of three
    double squareRoot = calculateSquareRoot(number);
    double powerOfThree = calculatePowerOfThree(number);

    // Print the results
    printf("The square root of %.2f is %.2f\n", number, squareRoot);
    printf("The number %.2f raised to the power of three is %.2f\n", number, powerOfThree);

    return 0;
}
```

Activity 1: Binary to Decimal Converter

1	0	0	1	1	0	1	1	₂
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
128	64	32	16	8	4	2	1	

Binary to Decimal Converter

Write a program that takes three integer type inputs `bin1`, `bin2`, and `bin3` from a user and prints the corresponding decimal value. `bin1`, `bin2`, and `bin3` must take either value `0` or `1`. Implement the following:

1. An `instruct` function with a brief description of the program.
2. Create a function that takes integer type inputs `bin1`, `bin2`, and `bin3` and computes the corresponding decimal value `dec` as $dec = bin3 * 2^2 + bin2 * 2^1 + bin1 * 2^0$.
3. Use top-down design to implement the main program that calls these functions based on user input.

Test Case 1:

- Input: `bin1 = 0`, `bin2 = 1`, `bin3 = 1`
- Output: 6

Test Case 2:

- Input: `bin1 = 1`, `bin2 = 0`, `bin3 = 1`
- Output: 5

Activity 2: Quadratic Equation Solver

$$ax^2 + bx + c = 0$$

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Quadratic Equation Solver

Write a program that solves quadratic equations of the form $ax^2 + bx + c = 0$ using functions. Implement the following:

1. An `instruct` function with a brief description of the program.
2. A function `calculateDiscriminant` that takes the coefficients `a`, `b`, and `c` as input arguments and returns the discriminant. Use the formula: $\text{discriminant} = b^2 - 4ac$.
3. A function `calculateRoots` that takes the coefficients `a`, `b`, `c`, and the discriminant as input arguments, and returns the roots $(-b \pm \sqrt{\text{discriminant}})/(2 * a)$ of the equation. The function should return the roots as output arguments.
4. Use top-down design to break down the problem and implement the main program that calls these functions.

Test Case 1:

- Input: `a = 1`, `b = 0`, `c = 0`
- Outputs: `0` and `0`

Test Case 2:

- Input: `a = 1`, `b = 3`, `c = 1`
- Outputs: `-0.382` and `-2.618`

Lab Submission Guidelines

- Compile a **small report including screenshots showing you work**
- **Rename the report** with **your IT number** and **covert it to a pdf** file (Example ITxxxxxxxxx.pdf)
- Upload the pdf file to the GitHub Classroom

